

Je commence par la PRÉSENTATION PERSONNELLE

- **J'ai 37** ans et suis originaire de Montpellier (je vous prie de m'excuser si je semble fatigué, j'ai fait 12 heures de bus et je suis parti hier à 16h de Montpellier)
- J'ai été **Commercial** pendant 11 ans et **Boulangier** de mes 20 ans à 23 ans
- **J'ai toujours été Passionné** d'informatique notamment des montages photo, vidéo, la création de logos, la gestion de forums ... mais jamais de développement
- J'ai donc eu l'envie de compléter mes compétences et de faire de **ma passion** un projet de **reconversion**
- **En 2016** j'ai intégré cette communauté de F1 et j'ai rapidement participé à son organisation
- **Récemment En 2024**, Je me suis engagé en tant que commissaire de piste bénévole au Circuit Paul Ricard pour devenir acteur de ma passion

Ensuite, je vous présente CETTE COMMUNAUTE DE PASSIONNES DE F1 :

- Elle a été créé en **2008** sur PS3 à l'époque et en est à sa 26^{ème} saison actuellement
- Le championnat est séparé en **2 catégories** F1 et F2
- **Les courses sont organisées** en ligne, deux soirs par semaine, avec le jeu **F1** sur **PlayStation 5**
- **Et les joueurs sont quasiment tous équipés** d'un casque-micro, d'un volant et d'un playseat

Les BESOINS, qui ont permis la mise en place de ce projet, répondent à plusieurs problématiques :

- Les classements sont gérés **manuellement** (fichiers Excel et images statiques)
- Le processus est **long**, répétitif et sujet à des **erreurs**
- La consultation des résultats est **peu lisible**
- La communauté est gérée uniquement sur **Discord**
- **Et n'est donc pas référencé en ligne ce qui** rend difficile le **recrutement** de nouveaux pilotes

Les ENJEUX identifiés de ce projet sont :

- De Créer d'une **base de données** dynamique
- **D'Automatiser** le calcul de ces classements **et de Centraliser** ces informations
- Et enfin d'Offrir une vitrine **SEO** à la communauté

Le cahier des charges comprend un PLAN D'ARCHITECTURE avec :

- Une Page **Accueil** (optimisée SEO avec présentation, liens réseaux sociaux, lien nous rejoindre vers le Discord)
- Une Page **Classements** avec Palmarès et Statistiques où toutes les données enregistrées en BDD sont centralisées
- Une Page **Mentions légales** qui se conforme à la réglementation en vigueur
- **Et un Dashboard** sécurisé pour gérer les résultats du championnat

Sur le PLAN FONCTIONNEL il faut :

- Créer une gestion des **rôles** pour les administrateurs, modérateurs et utilisateurs
- **Sécuriser** les formulaires et les sessions
- Respecter le **RGPD** et **l'accessibilité**

Pour le PLAN TECHNIQUE, le cahier des charges comprend l'utilisation :

- **De PHP** en architecture MVC pour le back-end
- **De MySQL** pour la base de données
- **Des langages HTML, CSS et JavaScript** pour le front-end (**voir l'annexe p.48 du schéma du parcours des requêtes sécurisées**)
- **Et du versioning** Git/GitHub

Sur le plan **TECHNIQUE**, les contraintes sont :

- L'architecture **MVC** avec les Modèles, Vues et Contrôleurs
- **La Sécurisation** contre les injections SQL et XSS, tokens CSRF
- **Le HTTPS** obligatoire
- Et une Base de données **MySQL** qui doit être cohérente

Pour les contraintes **FONCTIONNELLES**, elles sont :

- **L'automatisation** des calculs de classement
- La création d'une interface CRUD avec accès **sécurisé**
- Et l'affichage correct des **résultats** avec des **filtres de tris** pour les classements

Pour le **DESIGN ET L'ACCESSIBILITE**, il faut mettre en place :

- Un design **responsive** mobile-first
- Avoir un contraste et une lisibilité **optimisés**
- Et une **navigation clavier** grâce aux balises sémantiques et attributs ARIA.

Enfin les **LIVRABLES ATTENDUS** sont :

- **Le code source** complet
- La base de données **exportée** au format .sql
- **Le README** détaillé avec la présentation de l'application et la procédure de déploiement

Les environnements comprend d'abord l'ENVIRONNEMENT HUMAIN avec :

- **Une petite communauté** de 86 membres avec 3 administrateurs (dont je fais parti) et 6 modérateurs aidant à la gestion du championnat
- **Je reste seul** à réaliser ce projet mais en informant régulièrement les membres du staff à chaque étape de sa conception

L'ENVIRONNEMENT TECHNIQUE est le suivant :

- **L'utilisation de l'IDE** : Visual Studio Code
- **Le développement local avec** Wamp Server
- **Le Versioning** Git avec un dépôt GitHub public (**voir l'annexe p.60 du Lien QR Code vers GitHub**)
- **Les hébergements distants sont le** OVH de l'école pour les tests (**voir l'annexe p.60 du Lien QR Code vers le site**)
et O2SWITCH pour le déploiement final
- **La Base de données est en** MySQL administrée via phpMyAdmin
- **Les Langages** utilisés sont le HTML, CSS, JavaScript et PHP
- **Le Navigateur** exploité est Chrome avec ses outils intégrés : inspecteur et console
- **Et enfin le matériel** nécessaire est un Ordinateur personnel connecté à Internet avec un Double écran

Pour la CHARTE GRAPHIQUE de l'application :

- Je me suis inspiré du site officiel de la **Formule 1**
- **En utilisant les Couleurs similaires** et les polices **Google Fonts** proches
- C'est à ce moment là aussi que le choix des Formes **arrondies** pour les sections a été fait avec des border radius

Vous pouvez voir l'aperçu des codes couleurs utilisés:

- Noir pour **l'arrière-plan**
- Noir éclairci pour les **sections**
- Rouge pour les **boutons** et **en-têtes** de tableaux

Pour terminer l'élaboration de cette charte graphique, le choix des polices a été porté sur :

- **Titillium Web** pour le **contenu**
- **POPPINS** pour les **titres**

LES MAQUETTES

Ont été faite sur FIGMA comprenant :

- Un Zoning
- Des Wireframes
- Un Prototype

(voir le dossier p.6 et l'annexe p.38 des Liens QR Code vers Figma)

- **Egalement j'ai réalisé un Schéma** d'enchaînement des maquettes sur **GlooMaps (voir le dossier p.9)**
- **Cette étape m'a permis de simplifier au maximum** la navigation dans le but d'optimiser l'expérience utilisateur
- Ces modélisations ont ensuite permis de **valider** la pertinence des **choix UX / UI**
- Avant d'écrire la moindre ligne de code, cela a validé la bonne conception globale de l'application

Pour le RESPONSIVE DESIGN

Il a été construit en utilisant le HTML, le CSS et le JavaScript

(voir annexe p.43 sur le responsive design de la page Classements)

- Le HTML permet d'appeler les **classes** et différentes **feuilles de style CSS** :

style.css + <700px + <900px + <1400px

- Le **CSS**, avec les Feuilles de style propre à chaque type d'écran, définit le comportement des **classes**
et les **media queries** pour affiner certains détails (comme pour les écrans en dessous de **<500px**)
- Et enfin le **JavaScript m'a permis de** réduire la taille du nom des pilotes et des teams avec la fonction **substring(0,8)** adaptée selon la taille de l'écran
- Ce principe de responsive design a été appliqué à **toutes les pages**

Pour vous montrer un exemple d'INTERFACES STATIQUES

J'ai choisi de mettre en avant le fichier Views/base.php qui permet :

- La construction du **HTML commun** à toutes les pages avec la déclaration du doctype, les balises sémantiques, les métadonnées, les favicon, les feuilles de styles
- **La mise en place des Balises structurantes avec** header, nav, main et footer améliorant l'accessibilité et le référencement
- Le chargement du contenu du <main> selon les pages demandées qui se fait grâce à la variable **\$content**

(voir annexe p.39 sur le code de la page d'accueil)

Ce fichier Views/home/index.php construit, lui, l'affichage de la page d'accueil dans la variable \$content et comprend :

- **Des balises sémantiques** h1, h2, p, strong
- Du contenu optimisé **SEO** avec les mots clés adaptés au référencement
- Un bandeau animé en **CSS**

Pour les exemples d'INTERFACES DYNAMIQUES (voir dossier p.13 sur le code de la partie dynamique)

Ici nous pouvons voir le fichier `Views/dashboard/circuits/index.php` :

- Il permet l'accès à l'interface administrateur pour la gestion des circuits
- Ce fichier utilise la boucle **foreach** pour afficher dynamiquement chaque circuit dans le tableau
- L'utilisation de **htmlspecialchars()** permet la protection contre la faille XSS

Un autre exemple d'interface dynamique avec le fichier `Views/classements/standings.php`

(erreur de chemin 'dashboard' dans le dossier de projet p.14)

- Il permet l'affichage de la Page **Classements** visible par tous les visiteurs
- **Le Risque rencontré était que la** page soit trop longue à parcourir avec les Classement Pilotes, Constructeurs, les Pénalités et Les Résultats de chaque GP pour chaque catégorie
- Il fallait trouver une solution **dynamique** pour éviter cette surcharge
- C'est donc l'utilisation **d'AJAX** avec **fetch** pour le tableau des résultats de GP qui a permis d'avoir accès à tous les résultats complets sans avoir à recharger la page
- Au clic sur un des GP de la liste, un **modal** affiche les résultats détaillés du GP demandé grâce à la méthode GET qui récupère l'id
- **La conversion de type (int)** (ou Cast) permet d'éviter les injections SQL

PRÉSENTATION DE LA BASE DE DONNÉES

(voir dossier p.15 sur les schémas conceptuel, logique et physique)

Le MCD

- A été fait en suivant la Méthode **MERISE** grâce au logiciel gratuit **Looping**
- **Cela a permis de définir la Structure** globale de la BDD
- **Les Relations** entre les différentes **entités**
- Une Vue d'ensemble lisible
- **Et les Cardinalités** pour respecter les règles métier.
- Le **MLD** a été ensuite généré pour créer les relations avec les **clés primaires** et **clés étrangères**

Le MPD

- Dans Looping est généré aussi automatiquement
- **Il crée le Script SQL** avec les instructions CREATE TABLE **prêt à être importé dans phpMyAdmin**
- **Cela m'a permis le Déploiement** d'une base structurée et fiable

Pour présenter l'ARCHITECTURE MVC

Je vais vous expliquer le rôle de chaque lettre :

M pour les MODELS

Et l'Accès aux données

- Ils permettent de se connecter aux tables de la BDD
- Et d'effectuer les requêtes ou algorithmes des classements

V pour les VIEWS

Qui construisent les interfaces utilisateur

- Avec des Templates HTML CSS JavaScript
- Et des Vues instanciées par la méthode render du Controller

C pour les CONTROLLERS

Qui contiennent la Logique métier

- Le Controller principal, lui, centralise la sécurisation et permet l'affichage dans les vues avec la méthode render
- Et les Controllers enfants permettent le CRUD des tables de la BDD

Cette séparation claire des responsabilités permet de mieux comprendre et maintenir le code

J'en profite pour justifier l'absence des Entities avec les Getters et Setters, c'est un choix volontaire pour ne pas alourdir le code. Il est clair qu'avec un ORM, ils auraient été indispensable pour mieux contrôler l'accès aux données. Mais au vu de la taille du projet, j'ai préféré alléger mon code.

Ici nous allons voir un exemple de COMPOSANTS MÉTIERS
(voir dossier p.18 et annexe p.49 sur le code de composants métier)

Pour le CRUD des tables de la BDD, ils sont présents dans

- Les Contrôleurs **enfants**
- Ces contrôleurs enfants contiennent les méthodes gérant la **logique métier avec les méthodes :**
- **create()** **index()** **update()** **et** **delete()** qui permettent de
- Créer Lire Mettre à jour et Supprimer **les données**
- **Tous ces contrôleurs utilisent des Requêtes préparées** pour protéger des injections SQL

Enfin le CONTROLLER Principal quant à lui est

- Une classe abstraite dont **héritent** tous les contrôleurs enfants
- Elle ne contient pas de logique métier mais réuni les **fonctionnalités communes**
- La Méthode **authMiddleware** sécurise comme un pare-feu grâce aux méthodes de la classe **Auth**, présente dans le cœur de l'application.
- Enfin la Méthode **render, elle, permet de** générer les Vues

Concernant les COMPOSANTS D'ACCÈS AUX DONNÉES

(voir dossier p.20 et annexe p.56 sur le code de composants d'accès aux données)

Ici nous voyons le fichier Core/DbConnect.php

- Dont la classe **DbConnect** est présente dans le cœur de l'application
- Grâce à ça, la connexion est **centralisée**.
- **J'ai également utilisé PDO**, natif PHP, car il est performant, simple et sécurisé
- **Cette sécurité** a été mise en place avec les **attributs natifs PDO**
- **PDO permet également la maîtrise** totale des requêtes SQL. C'était donc plus simple qu'un ORM pour un projet de cette taille.

Tous les MODELS

- **Héritent** de la classe **DbConnect** pour instancier la connexion à la BDD
- Ils exécutent les **requêtes SQL**, les requêtes d'**algorithmes** et utilisent systématiquement les requêtes **préparées**.
- Le retour des données se fait sous forme d'objets grâce à **fetch** ou **fetchAll**

Ici nous voyons la gestion des SESSIONS & des AUTHENTIFICATIONS **(voir dossier p.22 sur les éléments de sécurité)**

Dans le fichier Core/Auth.php

- Dont la classe **Auth** est dans le cœur de l'application
- La Méthode **start** configure les **paramètres** de sécurité
- **Dont notamment le HTTPS** obligatoire
- Et la Durée de vie de la **session**

Les méthodes check & hasRole de cette classe permettent de

- Savoir si l'utilisateur est **connecté**
- Savoir si l'utilisateur possède un **rôle**
- **Mais surtout de centraliser** cette vérification d'authentification **Pour sécuriser** l'accès à l'application
- Tous les contrôleurs enfants en **bénéficient** grâce au Controller principal

Pour la SÉCURISATION DES MOTS DE PASSE

Elle se fait dans le fichier Controllers/UsersController.php,

dont le Contrôleur permet la gestion des **utilisateurs**,

et sa méthode create

Qui contient un password_hash pour stocker les mots de passe

Dans le fichier **Controllers/AuthController.php**,

dont le contrôleur gère la **connexion** et **déconnexion**,

sa méthode Login

Utilise ensuite password_verify pour la comparaison des mots de passe hachés

Avec la mise en place de ces méthodes, même en cas de **fuite**,

les mots de passe resteront **difficiles** à exploiter puisqu'ils ne sont jamais stockés en clair.

La sécurisation se fait également grâce aux REQUÊTES PRÉPARÉES

Mais avant cela les Contrôleurs

Utilisent la méthode **Form::validatePost** pour vérifier la présence des champs nécessaires

Ensuite prepare

Construit la requête sans l'exécuter et surtout sans passer directement les valeurs issues des formulaires

Et execute

Effectue la requête préparée avec les variables. Cette approche permet de se protéger des **injections SQL**

Pour finir sur la sécurité, nous voyons ici les PROTECTIONS XSS & l'utilisation des TOKENS CSRF

Pour les protections XSS

- C'est la Classe **Form**, présente dans le cœur de l'application,
- Qui construit les formulaires avec tous les champs et labels
- Et Utilise systématiquement **htmlspecialchars**
- De cette manière, l'échappement se fait dès la **construction** des formulaires et de manière centralisée

Même principe pour les TOKENS CSRF

- Grâce aux classes **Auth** & **Form**, présentes dans le cœur de l'application,
- Chaque **session** possède un **jeton**
- Et chaque **formulaire**, construit dans les Contrôleurs enfants, possède un jeton.
- La comparaison des tokens est faite ensuite grâce à la classe **Auth** : Si le token est manquant ou incorrect, la requête est **refusée**

Ce JEU D'ESSAI est à voir plus en détails (à la page p.26 du dossier)

Il met en avant

- Situation particulière mais courante : Un **pilote** ayant **changé d'équipe** en cours de saison
- L'Objectif de ce jeu d'essai est de vérifier la **logique métier** dans laquelle
le **Classements Pilotes** et le **Classement Constructeurs** applique correctement les points

Sans rentrer dans les détails, voici les :

DONNÉES EN ENTRÉE

DONNÉES ATTENDUES

DONNÉES EN SORTIE

En conclusion l'ANALYSE DES ÉCARTS permet de mettre en avant

- La prise en compte du **changement d'équipe** et la conservation des **points** précédemment attribués à son 1^{er} team
- **Ainsi on a pu vérifier le respect des règles métier.**
- Cette logique d'attribution des points est correcte et la gestion de l'historique des résultats est fiable.

Concernant la *VEILLE SUR LES VULNÉRABILITÉS DE SÉCURITÉ* (voir dossier p.29 sur la veille effectuée)

Voici le CONTEXTE de cet exemple précis :

- **La vulnérabilité** de sécurité a été identifiée durant le projet.
- **Lors de la Déconnexion**, la session était restée active.
- Et au clic sur **Dashboard**, l'interface était toujours accessible. On ne retombait pas sur le formulaire d'authentification.
- **Les Risques étaient multiples** : Divulcation, Contournement, Réutilisation des sessions ...

La SOLUTION a été apporté grâce aux Documentations consultées:

- **OWASP** (qui permet de comprendre et prévenir les vulnérabilités de sécurité)
- **Et PHP Manual** (qui décrit les fonctions et bonnes pratiques).
- J'ai donc pu modifier le **AuthController** et sa méthode **logout** avec notamment la mise en place de **session_regenerate_id**
- Après cette modification, la Session est désormais réellement **détruite et la faille** neutralisée

Les TESTS FONCTIONNELS MANUELS

Ont été MIS EN PLACE

- Car ce Projet de petite taille en **MVC** sans **framework** rendait moins pertinents les **Tests unitaires** automatisés.

Mais surtout sa mise en place aurait été complexe.

- J'ai donc réalisé manuellement les tests **fonctionnels**, de **sécurité** et **d'interface**

(dont le tableau complet est à voir en annexe p.58)

Enfin pour le DÉPLOIEMENT (voir dossier p.33 sur le déploiement)

Les différents ENVIRONNEMENTS utilisés ont été :

- **WampServer** en Développement local
- **OVH du CEFii** pour les tests
- **Et O2SWITCH** pour l'hébergement définitif **avec le nom de domaine acheté sur internet.bs**

Pour le PLANNING :

- La phase de développement et tests en local s'est faite sur la période de **Novembre à Décembre**
- La préproduction pour les tests a été faite en **Janvier**
- Et la mise en production pour son hébergement final se fera en **Février 2026**

Enfin le README comprend :

- **Une présentation** de l'application avec le **plan de déploiement** en local
- Les instructions détaillées pour **cloner** et **installer** le dépôt **GitHub** pour permettre à un développeur de **configurer rapidement** l'application

Pour CONCLURE

Ce projet m'a apporté des SATISFACTIONS comme

- La Mise en œuvre de toutes les **compétences acquises** durant cette formation
- La conception de l'application de **A à Z** sans **framework**
- Et surtout d'avoir maintenant une plateforme **opérationnelle** et réellement **utilisée**

Les DIFFICULTÉS principales ont été :

- **Le Responsive design** pour les tableaux riches en données
- La Veille quotidienne sur les **vulnérabilités** potentielles
- Et la Mise en place des mécanismes de **sécurité**

Concernant les EVOLUTIONS à venir :

- La Plateforme continuera **d'évoluer** selon les besoins et les retours de la communauté
- Il y aura un suivi des commits sur **GitHub** pour continuer à partager ce travail
- **Et enfin travailler sur les idées de développement à venir** qui peuvent être par exemple de mettre en place une communication avec une API, permettre la publication d'articles, de commentaires ...

Je tenais également à remercier :

- **Olivier**, mon formateur en développement, pour son **accompagnement** tout au long du parcours de formation
- L'École du **CEFii d'Angers** pour leur soutien et leur encadrement rassurant
- Grâce à eux, cette formation à distance, qui était intimidante au départ, a été une révélation et m'a aidé à développer mon autonomie.
- Débuter la formation **sans connaissances** en développement et la finir avec un **projet finalisé** et **opérationnel** est le plus bel aboutissement de mon année.